

CYBERSECURITY ENGINEERING – FINAL ASSIGNMENT

SecureCorp Zero-Trust Access Control System

1. Module Information

Course: Advanced Cybersecurity

Level: 4th Year Engineering

Type: Practical Project (Group of 2)

Duration: 2 weeks

2. Assignment Overview

You are required to design and implement a secure distributed system for SecureCorp using:

- Kerberos-inspired authentication
- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)
- Policy-based authorization engine
- Zero Trust security model

3. Scenario Description

SecureCorp manages sensitive internal data across departments:

HR, Finance, IT, Operations.

All services operate in a distributed microservices architecture with strict security enforcement.

4. Learning Objectives

- Design authentication systems
- Implement RBAC and ABAC

- Build policy engines
- Simulate and mitigate attacks

5. System Architecture Requirements

Components:

1. Authentication Server (KDC)

- Verifies credentials
- Issues TGT and Service Tickets
- Tickets include identity, attributes, expiration, session key
- Must be encrypted and tamper-resistant

2. Policy Decision Point (PDP)

- Evaluates access requests
- Uses external policies (JSON)
- Returns ALLOW/DENY decisions

3. Resource Servers

- Validate tickets
- Extract identity
- Query PDP before granting access

6. Authentication Requirements

Steps:

1. Login TGT
2. Request Ticket Service Ticket
3. Service Access with Ticket + Authenticator

Security:

- Encryption
- Expiration validation
- Replay protection
- Mutual authentication (bonus)

7. Authorization Model

RBAC:

Roles:

Admin, Manager, Employee

Permissions:

Admin: Read, Write, Delete

Manager: Read, Write

Employee: Read

Advanced RBAC:

- Role hierarchy
- Separation of Duties

ABAC:

User attributes: department, role, clearance, location

Resource attributes: department, classification (public, confidential, secret)

Environment: time, IP (optional)

Policies:

- Department isolation
- Classification restrictions
- Time-based access (08:00–18:00)
- Clearance validation
- External access restrictions

8. Policy Engine

Policies must be external (JSON format)

Example:

```
{  
  "effect": "deny",  
  "condition": {  
    "resource.classification": "secret",  
    "user.location": "external"  
  }  
}
```

9. Security Analysis & Attack Simulation

Required attacks:

- Replay attack
- Ticket tampering
- Privilege escalation

- Unauthorized access

For each:

- Explain vulnerability
- Demonstrate exploit
- Implement mitigation

10. API Requirements

Authentication:

POST /login

POST /request-ticket

Resources:

GET /resource/:id

POST /resource

DELETE /resource/:id

11. Logging & Monitoring

- Authentication attempts
- Authorization decisions
- Policy evaluation
- Suspicious activity

12. Testing Requirements

- Valid scenarios

- Invalid scenarios
- Attack demonstrations

13. Deliverables

1. Source code
2. Technical report (10–15 pages)
3. Demo (video or live)

14. Bonus

- Mutual authentication
- Docker deployment
- External policy engines
- Advanced logging

15. Evaluation Criteria

Architecture: 20%

Authentication: 20%

RBAC + ABAC: 20%

Policy Engine: 15%

Security Analysis: 15%

Code Quality: 10%

16. Academic Integrity

Original work required. Plagiarism results in zero.

17. Final Note

This project simulates real-world secure system design and requires strong technical and analytical skills.